

Pontificia Universidad Javeriana
Fundamentos de Ingeniería de Software

DriveControl / AutoMinder Enterprise

Equipo SYNTIX TECH

Milestone 2 - Gestión básica de flota

Profesor: Luis Gabriel Moreno Sandoval
Fecha: 19 de mayo de 2026
Repositorio: https://github.com/puj-course/FIS_2610_3517_G4

Índice

1. Resumen ejecutivo del milestone	2
2. Objetivos del milestone	2
3. Alcance funcional	2
4. Evidencia ágil	2
5. Métricas del milestone	3
6. Evaluación del producto	3
7. Evaluación del proceso	3
8. Evaluación del esfuerzo	3
9. Postmortem del milestone	4
9.1. Qué salió bien	4
9.2. Qué salió mal	4
9.3. Causas raíz	4
9.4. Impacto	4
9.5. Acciones correctivas	4
9.6. Acciones preventivas y responsables sugeridos	4
10.Retrospectiva estrella de mar	4
10.1. Comenzar a hacer	4
10.2. Más de	4
10.3. Seguir haciendo	5
10.4. Menos de	5
10.5. Dejar de hacer	5
11.Plan de mejora del siguiente ciclo	5
12.Conclusión del milestone	5
13.Anexos o evidencias pendientes	5

1. Resumen ejecutivo del milestone

El Milestone 2 consolidó la gestión básica de flota con interfaces de conductores y vehículos, validación RUNT, patrones de diseño y pruebas de flujos críticos. GitHub live registra 37 issues cerradas, 14 HU cerradas estrictas y 1 PR asociado.

2. Objetivos del milestone

- Implementar flujos de conductores y vehículos.
- Integrar validación RUNT simulada.
- Aplicar separación de responsabilidades y patrones.
- Fortalecer pruebas funcionales de flota.

3. Alcance funcional

El alcance cubre gestión básica de flota, validaciones, componentes visuales y soporte de pruebas. No incluye cierre CI/CD, SMS ni dashboard final.

4. Evidencia ágil

Elemento	Valor	Evidencia
HU cerradas	14	Issues con patrón estricto de HU.
Issues cerradas	37	Milestone cerrado en GitHub.
PRs	1	PR #233 mergeado.
Commits	164	Ventana S5-S8 en Git local.

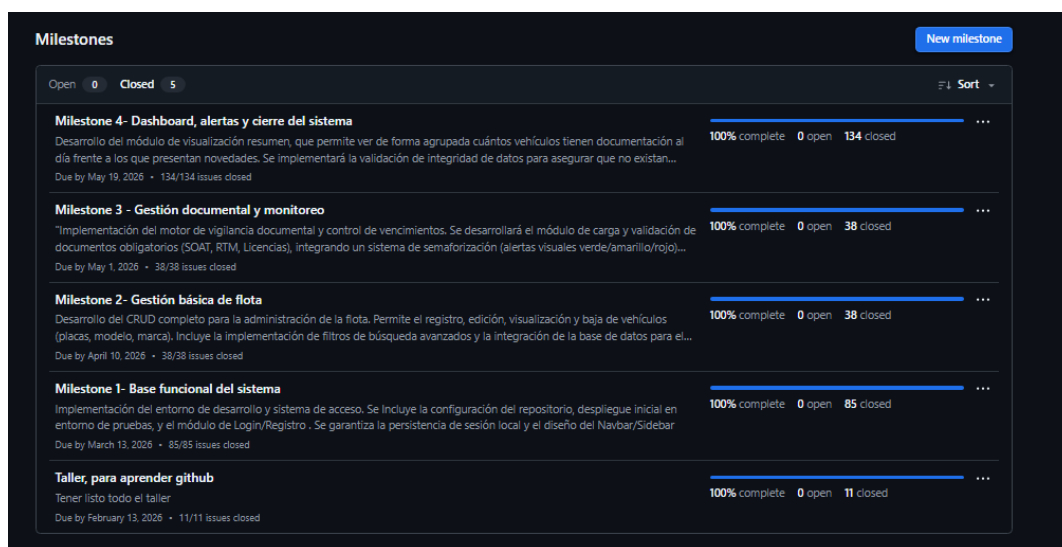


Figura 1: Milestones del semestre.

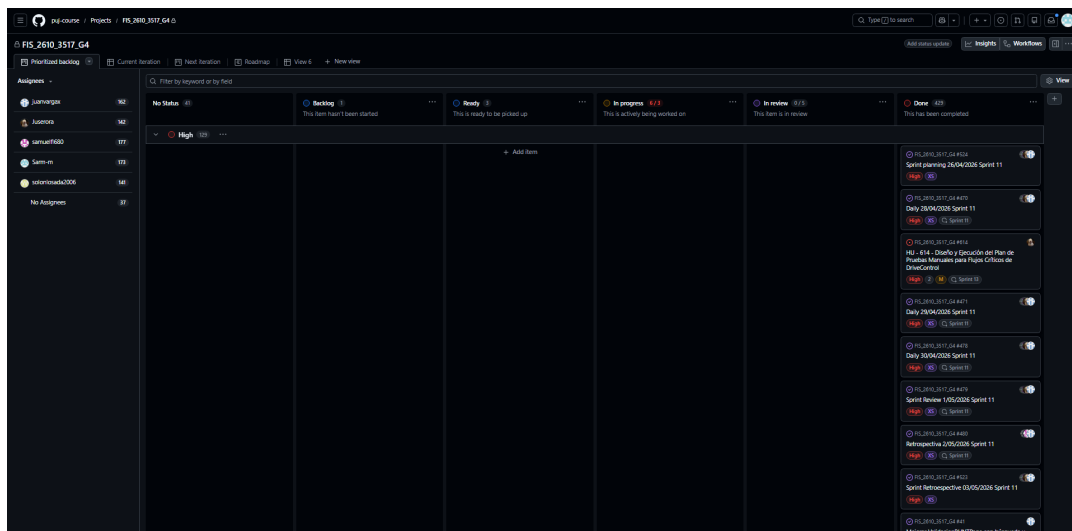


Figura 2: Tablero GitHub Project.

5. Métricas del milestone

Métrica	Valor
HU cerradas	14
Issues cerradas	37
Issues abiertas	0
Commits por ventana de sprint	164
PRs asociados	1
PRs mergeadas	1

Participación por issues asignadas: solonlosada2006 18, Sarm-m 9, Juserora 6, juanvargax 5 y samuelf1680 3.

6. Evaluación del producto

Se produjo gestión básica de flota, validación RUNT y pruebas de flujos críticos. El producto avanzó en mantenibilidad por separación de responsabilidades y reducción de código mezclado.

7. Evaluación del proceso

Mejoró la división en sub-issues, pero faltó mapa de dependencias e interoperabilidad entre entornos. GitHub sostuvo la trazabilidad del avance.

8. Evaluación del esfuerzo

El esfuerzo fue sostenido con 164 commits en S5-S8. La participación fue visible por issues asignadas, con carga fuerte en DevOps/flota y coordinación Scrum.

9. Postmortem del milestone

9.1. Qué salió bien

Se cerraron las issues, se fortaleció la arquitectura y se validaron flujos críticos.

9.2. Qué salió mal

Hubo problemas por nombres, alias, diferencias de entorno y componentes construidos antes de infraestructura base.

9.3. Causas raíz

La falta de convenciones de nombres y de entorno homogéneo causó bugs de importación e integración.

9.4. Impacto

El impacto fue retrabajo, menor visibilidad de integración y validaciones tardías.

9.5. Acciones correctivas

Estandarizar nombres, mapear dependencias y validar flujos en más de un entorno.

9.6. Acciones preventivas y responsables sugeridos

Configuration Manager debe definir convenciones; DevOps debe guiar entorno; QA debe mapear dependencias; Scrum Master debe cuidar sub-issues.

10. Retrospectiva estrella de mar

10.1. Comenzar a hacer

- Estandarizar convenciones de nombres mediante reglas compartidas.
- Definir rutas y menús antes de construir componentes dependientes.
- Recolectar evidencia técnica durante el sprint.

10.2. Más de

- Comunicar cambios en esquemas y contratos.
- Revisar dependencias técnicas en kick-off.
- Realizar commits frecuentes para reducir conflictos.

10.3. Seguir haciendo

- Dividir el trabajo en sub-issues atómicas.
- Aplicar patrones de diseño cuando reduzcan deuda real.
- Usar pruebas de flujos críticos.

10.4. Menos de

- Asumir compatibilidad entre entornos sin verificación.
- Posponer feedback visual hasta el cierre.
- Depender de datos manuales no reproducibles.

10.5. Dejar de hacer

- Construir componentes sobre infraestructura inexistente.
- Duplicar archivos sin revisar componentes existentes.
- Ignorar conflictos de alias como problemas individuales.

11. Plan de mejora del siguiente ciclo

Acción	Responsable	Evidencia esperada	Criterio de éxito
Definir convenciones	Configuration Manager	Regla o documento	Menos errores de importación
Mapear dependencias	QA Lead	Checklist	Menos retrabajo
Validar entornos	DevOps Engineer	Guía reproducible	Resultado consistente
Mantener sub-issues	Scrum Master	Issues enlazadas	Trazabilidad clara

12. Conclusión del milestone

M2 fue finalizado y fortaleció la flota. Su aprendizaje central fue que arquitectura y proceso necesitan convenciones y entornos homogéneos.

13. Anexos o evidencias pendientes

- Pendiente de captura: flujos RUNT por escenario.
- Pendiente de auditoría: relación exacta commit por HU.